TRƯỜNG ĐẠI HỌC VĂN LANG · KHOA CÔNG NGHỆ THÔNG TIN

BUỔI 7 · LÝ THUYẾT

Xây dựng module cốt lõi
Cấu trúc mã nguồn & quản lý phiên bản

Mở đầu Giai đoạn 2 – Hiện thực hóa

ĐỀ BÀI BÀI TẬP 2 · hạn nộp cuối buổi 12

Giảng viên chính HK261: **TS. Nguyễn Trí Hải / ThS. Nguyễn Minh Tân / ThS. Trần Công Thanh**

Học kỳ 1, năm học 2026 – 2027

NỘI DUNG

Nội dung buổi học

01

Cấu trúc module và tách trách nhiệm trong mã nguồn

Từ sơ đồ kiến trúc ở BT1 sang cấu trúc thư mục thật; ba lớp và ranh giới giữa chúng; quy ước đặt tên; dấu hiệu nhận biết trách nhiệm đặt sai chỗ.

02

Xử lý lỗi và validation ở từng lớp

Ba tầng validation; phân biệt lỗi người dùng với lỗi hệ thống; thông báo lỗi hữu ích; nguyên tắc không nuốt lỗi.

03

Git workflow cá nhân và quy ước commit

Nhịp commit theo đơn vị công việc; commit message có ý nghĩa; nhánh làm việc; xử lý khi commit nhầm secret; duy trì README liên tục.

04

Phần chuyên ngành – sinh viên tự học theo tài liệu

REST API với Node/Express hoặc Spring Boot [SE] · pandas và validation pipeline [DA] · thu thập và tiền xử lý dữ liệu [AI].

05

ĐỀ BÀI BÀI TẬP 2

Năm thành phần phải nộp, bảng tiêu chí kỹ thuật tối thiểu BT2, cách chấm bằng clone-and-run, hạn nộp cuối buổi 12.

ĐẶT VẤN ĐỀ – ĐIỀU KHÁC BIỆT CỦA GIAI ĐOẠN 2

Mã chạy được / Executable code

- Ba nguyên tắc chất lượng của học phần giữ nguyên, chỉ đổi đối tượng áp dụng.
  - TRUY VẾT ĐƯỢC: mỗi module phục vụ User Story nào.
  - TÁI LẬP ĐƯỢC: người khác clone repo và chạy được.
  - KIỂM CHỨNG ĐƯỢC: có test chạy pass, có ảnh chụp kết quả.
- Cách chấm BT2: Clone Repo của Sinh viên để chạy theo README.
- Câu hỏi quan trọng nhất suốt sáu buổi tới: **"người khác chạy lại các đoạn mã được không?".**
- Và một ràng buộc nữa: BT2 phải hiện thực ĐÚNG thiết kế đã nộp ở BT1.
  - Đổi hướng giữa chừng sẽ bị trừ ở tiêu chí Nhất quán & Tích lũy.

01 Cấu trúc module và tách trách nhiệm trong mã nguồn

Từ sơ đồ kiến trúc sang cấu trúc thư mục · ranh giới giữa các lớp · quy ước đặt tên

Sơ đồ kiến trúc ánh xạ một–một sang thư mục

MỖI LỚP TRONG SƠ ĐỒ = MỘT THƯ MỤC TRONG src/

```
Sơ đồ kiến trúc (BT1)              Cấu trúc thư mục (BT2)
--------------------------         ----------------------------------------
LỚP TRÌNH BÀY                      src/routes/     (hoặc src/web/)
  Trang tiếp nhận                    ticket.routes.js
  Kiểm tra đầu vào                   src/validators/ticket.validator.js

LỚP NGHIỆP VỤ  (Service)           src/services/
  TicketService                      ticket.service.js
  ClassifyService                    classify.service.js
  >>> quy tắc nghiệp vụ <<<         (QT-01, QT-03, QT-04, QT-05 nằm ở đây)

LỚP TRUY CẬP DỮ LIỆU               src/repositories/
  TicketRepository                   ticket.repository.js
  CustomerRepository                 customer.repository.js

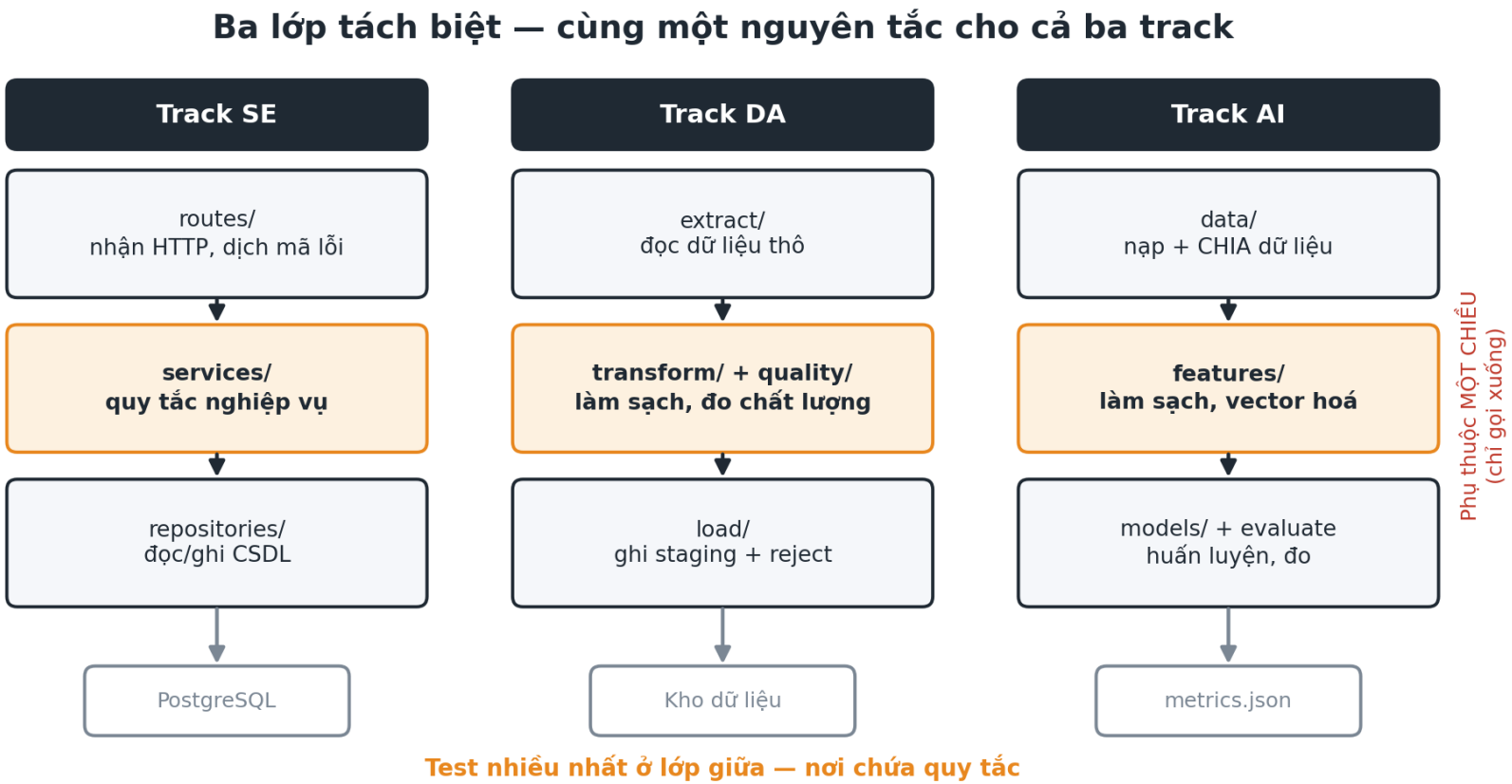
LỚP LƯU TRỮ                        migrations/  +  seeds/
  PostgreSQL: tickets, customers     001_create_customers.sql

                                   tests/
                                     ticket.service.test.js   <- test lớp nghiệp vụ
                                     ticket.repository.test.js
```

Ghi nhớ

- Nếu người đọc mở repo mà không đoán được sơ đồ kiến trúc, nghĩa là cấu trúc chưa phản ánh thiết kế.
- Mục kiểm chứng BT2: "cấu trúc thư mục tách biệt rõ, không dồn toàn bộ mã vào 1 file".
- Tên thư mục nên khớp tên lớp trong sơ đồ — đó là truy vết ở mức mã nguồn.
- Track DA và AI dùng cùng nguyên tắc, chỉ đổi tên lớp: xem slide phần chuyên ngành.
- Nếu ở BT1 em thiết kế 3 lớp mà BT2 chỉ có 1 thư mục src/ — đó là mất nhất quán.

Cùng một hình dạng: nhận vào · quy tắc · dữ liệu



Đọc hình thế nào

- Mũi tên chỉ đi XUỐNG: lớp dưới không bao giờ gọi ngược lớp trên.
- Lớp giữa (tô cam) chứa quy tắc nghiệp vụ — test nhiều nhất ở đây.
- SE: services/ · DA: transform/ + quality/ · AI: features/ — lớp này không đọc ghi CSDL, không biết HTTP.
- Câu hỏi buổi 8: "Lớp giữa nằm ở thư mục nào?" — trả lời bằng cách mở repo.

Bốn dấu hiệu cho thấy trách nhiệm đang nằm sai chỗ

1. **Câu truy vấn SQL nằm trong lớp trình bày hoặc lớp nghiệp vụ**
   Hậu quả: đổi cơ sở dữ liệu phải sửa khắp nơi; không viết unit test cho nghiệp vụ được vì phải có cơ sở dữ liệu thật. Cách sửa: đưa mọi truy vấn xuống lớp repository.

2. **Quy tắc nghiệp vụ nằm trong lớp repository hoặc trong câu truy vấn**
   Ví dụ: câu SQL có điều kiện "chỉ lấy phiếu còn bảo hành". Hậu quả: quy tắc bị giấu trong SQL, không ai đọc mã nghiệp vụ mà biết được. Cách sửa: đưa quy tắc lên lớp service.

3. **Lớp nghiệp vụ biết về HTTP: nhận request, trả về mã trạng thái**
   Ví dụ: hàm service nhận tham số req và trả về res.status(400). Hậu quả: không gọi lại được từ chỗ khác (script, tác vụ nền). Cách sửa: service nhận và trả dữ liệu thuần, controller lo chuyện HTTP.

4. **Lớp trình bày gọi thẳng xuống lớp truy cập dữ liệu, bỏ qua lớp nghiệp vụ**
   Thường xảy ra với các thao tác "chỉ đọc thôi mà". Hậu quả: quy tắc phân quyền và che dữ liệu nhạy cảm (QT-14, QT-15) bị bỏ qua. Cách sửa: mọi lối vào đều đi qua lớp nghiệp vụ.

Một route "làm tất cả" và cùng chức năng sau khi tách lớp

TRƯỚC — mọi thứ dồn trong route (sai)

```
r.post('/tickets', async (req, res) => {
  const { rows } = await db.query(        // (1) SQL
    `SELECT * FROM tickets WHERE serial_no=$1
     AND status NOT IN ('HOAN_TAT','HUY')`,
    [req.body.serialNo]);
  if (rows.length)                        // (3) quy tắc
    return res.status(409).send('trùng');  //     lẫn HTTP
  if (!req.body.priority)
    return res.status(400).send('lỗi');   // vô nghĩa
  await db.query('INSERT INTO tickets ...');
  res.status(201).send('ok');
});
```

SAU — mỗi lớp đúng một việc

```
// routes: chỉ dịch HTTP <-> nghiệp vụ
r.post('/tickets', async (req, res, next) => {
  try {
    const input = validateReceive(req.body);  // tầng 1
    const t = await ticketService.receiveDevice(input);
    res.status(201).json({ data: t });
  } catch (e) { traLoi(e, res, next); }   // 400/409/422
});
// services: quy tắc, không biết HTTP
if (openOne) throw new AppError('PHIEU_DANG_MO', ...);
// repositories: chỉ đọc ghi
findBySerial(sn) { return this.db.query(`SELECT ...`); }
```

Được gì: test QT-09 chạy trong vài mili giây, không cần máy chủ web lẫn CSDL · đổi PostgreSQL sang SQLite chỉ sửa repositories/ · thông báo lỗi có mã và lý do rõ ràng.

Đặt tên: bốn quy tắc thực dụng

1. Tên nói lên MỤC ĐÍCH, không nói cách hiện thực.

Nên: layPhieuQuaHan(). Tránh: selectTicketWhereDueDateLessThanNow().

2. Hàm là ĐỘNG TỪ, dữ liệu là DANH TỪ, hàm trả về đúng/sai bắt đầu bằng la/co/duoc.

taoPhieu(), danhSachPhieu, laConBaoHanh(), duocPhanCong().

3. Dùng đúng thuật ngữ trong bảng thuật ngữ của SRS.

Nếu SRS gọi "phiếu bảo hành" thì mã nguồn dùng ticket, không dùng lẫn request, order hay case. Đây là truy vết ở mức mã nguồn và được chấm.

4. Tên dài mà rõ tốt hơn tên ngắn mà mơ hồ.

Nên: phieuQuaHanTheoKyThuatVien. Tránh: data, temp, list2, x.

Tiếng Việt không dấu hay tiếng Anh đều được — miễn là NHẤT QUÁN trong toàn bộ repo. Trộn hai thứ mới là vấn đề.

Vì sao điều này được chấm

- Tiêu chí 4 của rubric (Tài liệu & Trình bày) chấm cả sự rõ ràng của mã nguồn.
- Ở buổi 18, có vấn đáp về mã nguồn. Tên biến mơ hồ khiến chính sinh viên cũng khó giải thích.
- Đây là kỹ năng nghề: trong công ty, mã của nhà phát triển được người khác đọc nhiều hơn người đó viết.

02 Xử lý lỗi và validation ở từng lớp

Ba tầng validation · lỗi người dùng và lỗi hệ thống · thông báo lỗi hữu ích · không nuốt lỗi

BA TẦNG VALIDATION – MỖI TẦNG CÓ TRÁCH NHIỆM RIÊNG

Cùng một dữ liệu được kiểm ở ba nơi, vì ba lý do khác nhau

1

Tầng 1 — Giao diện (trình bày)

Kiểm định dạng cơ bản để người dùng biết ngay.

Ví dụ: số điện thoại phải đủ 10 chữ số.

Mục đích: trải nghiệm tốt.

KHÔNG được tin tưởng — người dùng có thể gọi thẳng API.

2

Tầng 2 — Nghiệp vụ (service)

Kiểm quy tắc nghiệp vụ. Đây là tầng QUAN TRỌNG NHẤT.

Ví dụ: QT-03 thiết bị phải thuộc về khách; QT-05 còn bảo hành hay không.

Mục đích: bảo đảm đúng nghiệp vụ.

BẮT BUỘC phải có.

3

Tầng 3 — Cơ sở dữ liệu (lưu trữ)

Ràng buộc toàn vẹn cuối cùng.

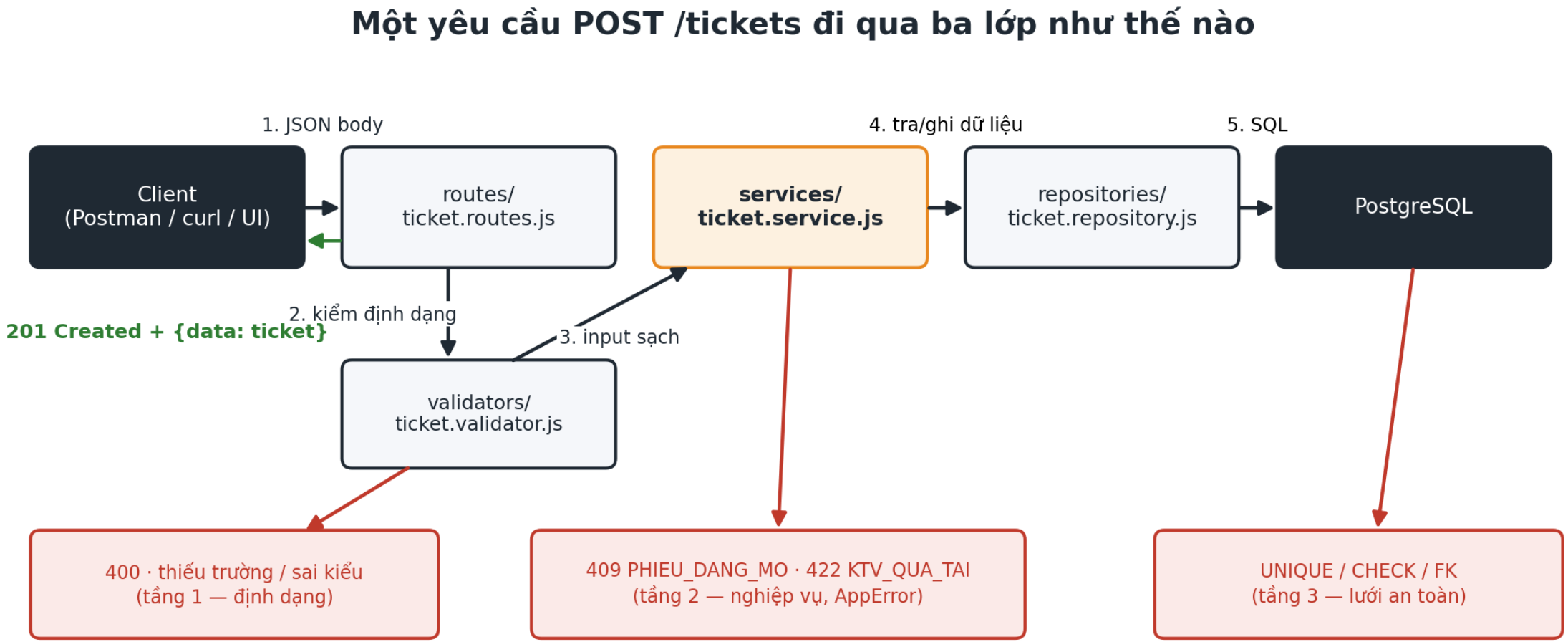
Ví dụ: NOT NULL, UNIQUE, FOREIGN KEY, CHECK.

Mục đích: dữ liệu không bao giờ hỏng dù mã nguồn có lỗi.

Là lưới an toàn, không thay được tầng 2.

Vì sao cần cả ba: tầng 1 có thể bị bỏ qua (gọi API trực tiếp); tầng 3 chỉ báo lỗi kỹ thuật khó hiểu và không diễn đạt được quy tắc phức tạp. Chỉ tầng 2 vừa bắt buộc vừa diễn đạt được nghiệp vụ. Nếu chỉ làm được một tầng vì thiếu thời gian, hãy làm tầng 2.

Mỗi loại lỗi bị chặn ở đúng một tầng



Đọc từ trái sang phải: lỗi định dạng dừng ở validator (400) · lỗi nghiệp vụ dừng ở service (409/422) · ràng buộc CSDL là lưới an toàn cuối · chỉ routes biết mã HTTP.

Quy tắc QT-05 được hiện thực như thế nào

QT-05: thiết bị còn bảo hành nếu (ngày tiếp nhận - ngày mua) <= số tháng bảo hành

```
// src/services/ticket.service.js
// Mỗi quy tắc nghiệp vụ là MỘT HÀM RIÊNG, tên hàm nhắc lại quy tắc.
function kiemTraConBaoHanh(thietBi, ngayTiepNhan) {          // QT-05
  if (!thietBi.ngayMua) {
    // Luồng ngoại lệ 3a đã đặc tả ở BT1: không có ngày mua
    // -> KHÔNG tự quyết định, đánh dấu cho quản lý phê duyệt
    return { conBaoHanh: null, lyDo: 'CHUA_XAC_MINH_BAO_HANH' };
  }
  const soThang = chenhLechThang(thietBi.ngayMua, ngayTiepNhan);
  const con = soThang <= thietBi.soThangBaoHanh;
  return { conBaoHanh: con, lyDo: con ? 'CON_BAO_HANH' : 'HET_BAO_HANH' };
}

async function taoPhieu(duLieu) {
  const khach = await customerRepo.timTheoId(duLieu.customerId);
  if (!khach) throw new LoiNghiepVu('KHONG_TIM_THAY_KHACH',
                                    'Không tìm thấy khách hàng');
  const thietBi = await deviceRepo.timTheoId(duLieu.deviceId);
  if (thietBi.customerId !== khach.id)                        // QT-03
    throw new LoiNghiepVu('THIET_BI_KHONG_THUOC_KHACH',
                          'Thiết bị không thuộc về khách hàng này');
  const bh  = kiemTraConBaoHanh(thietBi, duLieu.ngayTiepNhan);      // QT-05
  const han = tinhHanCamKet(duLieu.ngayTiepNhan, duLieu.mucUuTien);  // QT-04
  return ticketRepo.luu({ ...duLieu, conBaoHanh: bh.conBaoHanh, hanCamKet: han });
}
```

Ghi nhớ

- Ghi MÃ QUY TẮC (QT-03, QT-04, QT-05) ngay trong mã nguồn — đó là truy vết từ SRS tới code.
- Mỗi quy tắc là một hàm riêng, đặt tên nhắc lại quy tắc. Dễ test, dễ đọc, dễ sửa.
- Trường hợp "không có ngày mua" chính là luồng ngoại lệ đã đặc tả ở BT1 — hiện thực đúng nó.
- Mỗi hàm quy tắc như thế này chuyển thẳng thành một unit test ở BT2 và một test case ở BT3.
- LoiNghiepVu là lớp lỗi riêng, phân biệt với lỗi hệ thống — xem slide sau.

PHÂN BIỆT HAI LOẠI LỖI – XỬ LÝ KHÁC NHAU HOÀN TOÀN

Lỗi người dùng và lỗi hệ thống

- **LỖI NGƯỜI DÙNG (đã lường trước, là một phần của nghiệp vụ):**
- Nhập thiếu trường bắt buộc; thiết bị không thuộc về khách; thiết bị hết bảo hành; xuất linh kiện vượt tồn kho.
- Cách xử lý: trả về thông báo RÕ RÀNG cho người dùng, nêu đúng trường nào sai và sai thế nào. Không ghi log mức lỗi. Đây không phải sự cố.
- **LỖI HỆ THỐNG (ngoài dự kiến, là sự cố cần điều tra):**
- Mất kết nối cơ sở dữ liệu; hết bộ nhớ; lỗi lập trình như gọi thuộc tính của giá trị rỗng.
- Cách xử lý: GHI LOG đầy đủ kèm ngữ cảnh để điều tra; trả về thông báo chung chung cho người dùng (không lộ chi tiết kỹ thuật); không để hệ thống ở trạng thái dở dang.
- **NGUYÊN TẮC KHÔNG NUỐT LỖI: không bao giờ bắt lỗi rồi bỏ qua im lặng. Nếu chưa biết xử lý thế nào, hãy để lỗi nổi lên — im lặng nuốt lỗi là cách chắc chắn nhất để có một hệ thống chạy sai mà không ai biết.**

Thông báo lỗi tốt gồm ba phần

- CHUYỆN GÌ đã xảy ra: "Không thể tạo phiếu bảo hành".
- VÌ SAO: "Thiết bị có số serial SN123456 không thuộc về khách hàng Nguyễn Văn An".
- LÀM GÌ TIẾP: "Kiểm tra lại số serial, hoặc chọn Tạo thiết bị mới cho khách hàng này".
- Tránh: "Có lỗi xảy ra", "Error 500", "Invalid input", hoặc in nguyên vẹn thông báo lỗi kỹ thuật của cơ sở dữ liệu ra màn hình người dùng.
- Mục kiểm chứng C7 của BT2: dữ liệu sai bị từ chối với thông báo rõ ràng — không sập, không trả lỗi thô của thư viện.

Phản hồi lỗi chưa tốt và phản hồi lỗi tốt

CHƯA TỐT — lộ cấu trúc bên trong, không biết sửa gì

```
HTTP/1.1 500 Internal Server Error
{"error": "duplicate key value violates
 unique constraint \"tickets_ticket_code_key\"",
 "stack": "error: at Parser.parseErrorMessage
 (node_modules/pg-protocol/dist/parser.js:287)..."}
```

TỐT — mã nghiệp vụ + lý do + trường sai

```
HTTP/1.1 400 Bad Request
{"error": "DU_LIEU_KHONG_HOP_LE",
 "message": "Dữ liệu vào không hợp lệ ở các trường: priority",
 "chi_tiet": [{"truong": "priority",
   "ly_do": "phải là CAO, TRUNG_BINH hoặc THAP"}]}
```

Mã nghiệp vụ (service ném)	HTTP (routes dịch)	Khi nào	Người dùng cần làm gì
DU_LIEU_KHONG_HOP_LE	400 Bad Request	Thiếu trường, sai kiểu, sai định dạng ngày	Sửa đúng các trường được liệt kê
KHACH_KHONG_TON_TAI	404 Not Found	customerId không có trong CSDL	Chọn lại khách hàng hoặc tạo mới
PHIEU_DANG_MO	409 Conflict	Serial đã có phiếu chưa đóng (QT-09)	Mở phiếu đang xử lý thay vì tạo mới
(lỗi không lường trước)	500 Internal Server Error	Mất kết nối CSDL, lỗi lập trình	Thử lại sau; hệ thống GHI LOG để điều tra

03 Git workflow cá nhân và quy ước commit

Nhịp commit theo đơn vị công việc · nhánh làm việc · xử lý khi lỡ commit secret · README liên tục

Commit khi nào và commit cái gì

1 Một commit = một đơn vị công việc hoàn chỉnh

Đơn vị công việc là thứ em mô tả được trong một câu và hệ thống vẫn chạy được sau đó.

Ví dụ tốt: "thêm endpoint POST /tickets và validation trường bắt buộc".

Ví dụ xấu: một commit chứa cả sửa lỗi, thêm tính năng và đổi tên biến.

2 Commit đều theo tiến độ, không dồn

Nhịp hợp lý cho học phần này: 2–4 commit mỗi buổi làm việc.

Dồn 30 commit vào đêm trước hạn nộp sẽ bị trừ điểm — lịch sử Git cho thấy rõ điều đó.

Commit cũng là điểm lưu: hỏng thì quay lại được.

3 Không commit thứ sinh ra tự động

Không commit: node_modules/, venv/, __pycache__/, dist/, build/, file dữ liệu lớn, file mô hình đã huấn luyện.

Kiểm bằng git status trước mỗi commit.

Nếu lỡ commit rồi: thêm vào .gitignore rồi chạy git rm -r --cached <thư mục>.

Ví dụ thông điệp chưa tốt và tốt

Chưa tốt	Tốt
update	feat: thêm endpoint POST /tickets kèm validator
fix bug	fix: QT-09 cho phép tạo phiếu mới khi phiếu cũ đã HUY
code xong phần 2	test: thêm TC-02 giá trị biên ngày hết bảo hành
final final v2	docs: README bổ sung lệnh chạy migration và seed
sửa linh tinh	refactor: tách kiểm tra QT-05 thành hàm isUnderWarranty

feat tính năng · **fix** sửa lỗi · **test** thêm test · **docs** tài liệu · **refactor** sắp xếp lại mã, không đổi hành vi · **chore** việc phụ trợ (gitignore, cấu hình).

Tiêu chí 4 của rubric BT2 trừ điểm commit kiểu "update", "fix". Thông điệp tốt cho người chấm biết sinh viên làm gì mà không phải mở mã.

git log --oneline — một tuần làm việc tốt

```
9c1e2ab docs: README thêm lệnh test
71d03f4 test: TC-03 serial đang mở bị từ chối
5b8a210 feat: QT-09 chặn phiếu mở trùng
e42c9d1 test: TC-02 biên ngày hết hạn BH
3f6b7c0 feat: QT-05 kiểm tra hạn bảo hành
a01d5e9 feat: repository tickets + migration
0c9f8b2 chore: khung thư mục, .gitignore

# Mỗi dòng: một việc, đọc là hiểu,
# hệ thống vẫn chạy sau mỗi commit.
```

Xử lý ba tình huống thường gặp

CHÉP LẠI VÀ DÙNG KHI CẦN — không cần học thuộc

```
# 1. LỠ COMMIT FILE KHÔNG NÊN COMMIT (node_modules/, venv/, dữ liệu lớn)
echo "node_modules/" >> .gitignore
git rm -r --cached node_modules            # bỏ khỏi theo dõi, GIỮ lại trên đĩa
git commit -m "chore: bỏ node_modules khỏi theo dõi và bổ sung gitignore"

# 2. LỠ COMMIT SECRET (mật khẩu, khoá API, file .env thật)
#    >>> VIỆC ĐẦU TIÊN: ĐỔI KHOÁ NGAY LẬP TỨC <<<
#    Xoá ở commit sau KHÔNG đủ - lịch sử Git vẫn còn và vẫn đọc được.
echo ".env" >> .gitignore
git rm --cached .env
git commit -m "chore: bỏ .env khỏi theo dõi"
#    Nếu repo public và khoá còn hiệu lực: đổi khoá là BẮT BUỘC, không phải tuỳ chọn.

# 3. COMMIT NHẦM MESSAGE (chưa push)
git commit --amend -m "feat(ticket): thêm validation trường bắt buộc"

# 4. MUỐN XEM MÌNH ĐÃ LÀM GÌ - dùng trước khi nộp bài
git log --oneline -20                      # 20 commit gần nhất
git log --oneline --stat -5                # kèm danh sách file đã đổi
git diff HEAD~3 --stat                     # đổi gì trong 3 commit gần nhất

# 5. TẠO TAG KHI NỘP BÀI  (BT2 yêu cầu tag v0.2)
git tag -a v0.2 -m "Nộp Bài tập 2: module cốt lõi chạy được"
git push origin main && git push origin v0.2   # push nhánh KHÔNG đẩy tag
```

Ghi nhớ

- Tình huống 2 là tình huống nghiêm trọng nhất. Đổi khóa là việc ĐẦU TIÊN, trước cả việc sửa repo.
- Lệnh ở mục 4 rất hữu ích trước khi nộp bài: xem lại lịch sử để tự đánh giá nhịp commit của mình.
- BT2 yêu cầu tag v0.2. Không tạo tag thì thầy không biết chấm phiên bản nào.

04 Phần chuyên ngành và đề bài Bài tập 2

Stack hiện thực theo track · năm thành phần BT2 · bảng tiêu chí kỹ thuật tối thiểu

Hiện thực module cốt lõi theo chuyên ngành

Tài liệu tự học buổi 7 theo track có hướng dẫn chi tiết và ví dụ mã mẫu hoàn chỉnh trên luồng L2 / L6 / L10. Giảng viên chỉ nêu bối cảnh và đối chiếu checklist.

SE REST API + CSDL quan hệ

- Dựng project: Node/Express hoặc Spring Boot; kết nối CSDL bằng ORM
- Migration tạo bảng theo đúng DDL đã nộp ở BT1
- CRUD cho ≥ 2 entity, tách đủ ba lớp routes / service / repository
- Validation ba tầng; xác thực JWT ở mức cơ bản
- Unit test cho lớp service, thay repository bằng bản giả (mock)

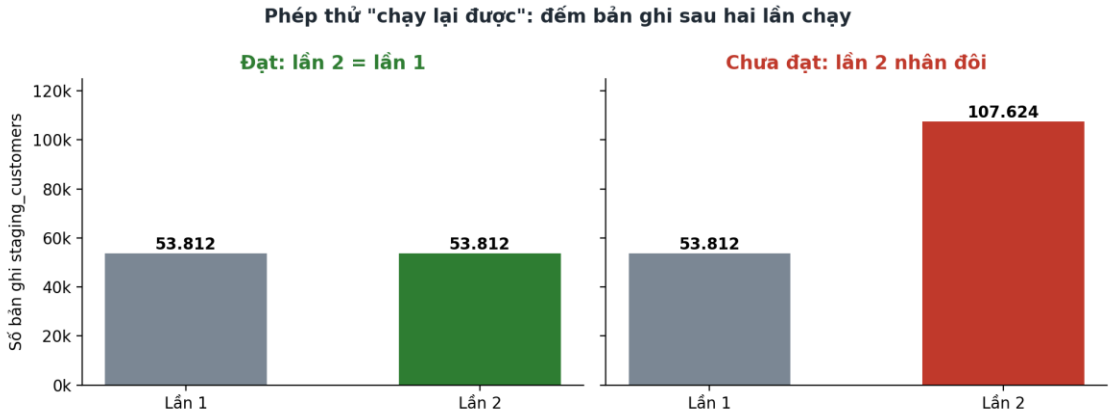
DA ETL + kiểm tra chất lượng

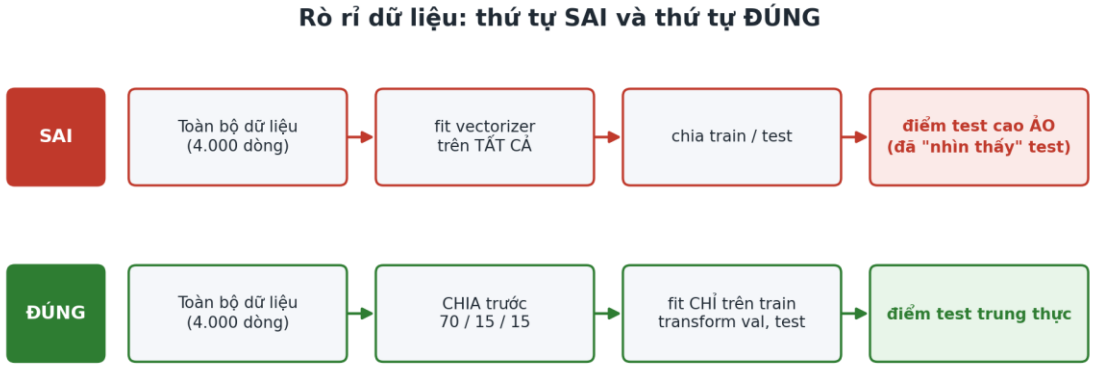
- Đọc nguồn bằng pandas: xử lý ba định dạng ngày, số điện thoại bốn dạng
- Hàm kiểm chất lượng riêng cho từng quy tắc CL1..CL5, mỗi hàm trả về số đo được
- Nạp vào bảng staging; ghi bảng reject cho dòng không xử lý được, kèm lý do
- Bảo đảm chạy lại được: chạy lần hai không nhân đôi dữ liệu
- Script chạy bằng MỘT lệnh, không phải notebook bấm từng ô

AI Thu thập + tiền xử lý dữ liệu

- Nạp issue_descriptions.csv; kiểm tra phân bố nhãn trước khi làm gì khác
- Làm sạch văn bản: chuẩn hoá chữ thường, bỏ thông tin định danh, GIỮ dấu tiếng Việt
- CHIA DỮ LIỆU TRƯỚC, sau đó mới fit vectorizer trên tập train
- Notebook EDA riêng; script huấn luyện riêng — notebook không phải sản phẩm cuối
- Baseline bắt buộc: DummyClassifier theo lớp đa số, để so sánh

"Chạy lại được" và "không rò rỉ dữ liệu"





[DA] Chạy pipeline hai lần với cùng đầu vào rồi đếm bản ghi. Lần hai nhân đôi nghĩa là load chỉ INSERT — sửa bằng "xoá rồi nạp lại theo batch_id" hoặc INSERT ... ON CONFLICT trên khoá nghiệp vụ.

GV sẽ hỏi: "Chạy lại lần hai thì số bản ghi thế nào?"

[AI] Khác biệt duy nhất là VỊ TRÍ bước chia dữ liệu. fit vectorizer trước khi chia thì điểm test đẹp hơn nhưng không thật — đó là vấn đề liêm chính, không chỉ là lỗi kỹ thuật.

GV sẽ hỏi: "Dòng nào trong train.py gọi fit, và nó chạy trên tập nào?"

BT2 · 20% ĐIỂM HỌC PHẦN · HẠN NỘP CUỐI BUỔI 12 · TAG v0.2

Năm thành phần phải nộp

1

Module cốt lõi chạy được — trọng tâm của bài

[SE] ≥ 2 thực thể đủ CRUD, tách routes / services / repositories, ≥ 2 quy tắc nghiệp vụ thật, migration + seed · [DA] pipeline một lệnh, ba pha E/T/L, ≥ 5 hàm kiểm chất lượng, bảng reject, nạp lại không nhân đôi · [AI] script huấn luyện một lệnh, chia dữ liệu TRƯỚC khi fit, có baseline, cố định hạt giống (seed).

2

Bộ kiểm thử tối thiểu ba trường hợp, thuộc ba loại khác nhau

Luồng đúng · giá trị biên · dữ liệu sai. Chạy bằng MỘT lệnh ghi trong README, PASS toàn bộ khi nộp; test tự dựng dữ liệu hoặc dùng đối tượng giả, không phụ thuộc dữ liệu trên máy.

3

Module mở rộng và phần trình bày kết quả

[SE] ≥ 3 màn hình gọi API thật qua lớp client riêng · [DA] ≥ 2 biểu đồ, mỗi biểu đồ trả lời một câu hỏi phân tích · [AI] ≥ 2 mô hình + baseline, chỉ số theo từng nhãn. Xử lý đủ ba trạng thái: đang tải · rỗng · lỗi.

4

Hệ thống tích hợp chạy được bằng lệnh — người khác dựng lại được

README mỗi bước một lệnh; tệp khoá phiên bản đã commit; .env.example đủ biến; có dữ liệu mẫu. Dockerfile / docker-compose (≤ 2 lệnh) là điểm cộng và là bước chuẩn bị cho BT3.

5

Tài liệu và phụ lục bắt buộc

README.md · docs/thay-doi-thiet-ke.md · docs/ai-disclosure.md (có cột "đã kiểm chứng thế nào") · .env.example. Nộp 1 file PDF + link repo có tag v0.2 trước khi hết buổi 12.

Ba loại kiểm thử bắt buộc — ví dụ theo track

Loại	Track SE	Track DA	Track AI
1. Luồng đúng	Thiết bị còn bảo hành thì tạo được phiếu	Tệp nguồn đúng định dạng thì nạp đủ số dòng	Mô hình dự đoán được với một mô tả hợp lệ
2. Giá trị biên	Ngày cuối còn bảo hành và ngày đầu tiên hết bảo hành	Dòng có giá trị đúng bằng ngưỡng cho phép	Mô tả ở độ dài tối thiểu cho phép
3. Dữ liệu sai	Thiếu trường bắt buộc → 400 kèm tên trường thiếu	Ngày không đọc được → vào bảng reject kèm lý do	Mô tả rỗng → lỗi rõ ràng, không ném ngoại lệ thô

Ba yêu cầu về hình thức — thiếu một là không được tính

- Chạy bằng MỘT lệnh ghi trong README (npm test · pytest -q). Giảng viên chạy đúng lệnh đó.
- PASS toàn bộ tại thời điểm nộp; test đang FAIL mà không ghi chú lý do bị tính là chưa đạt.
- Không phụ thuộc dữ liệu có sẵn trên máy: test tự dựng dữ liệu hoặc dùng repository giả. Ba test cùng một loại chỉ được tính là một.

Bảng tiêu chí kỹ thuật tối thiểu của BT2

Mục kiểm chứng CHUNG – áp dụng cho mọi track	Mục kiểm chứng RIÊNG theo track
☐ C1. Hệ thống chạy được từ trạng thái ban đầu theo đúng README. (CỐT LÕI) ☐ C2. Tách lớp đúng: ba lớp ở ba nơi, không gọi chéo sai chiều. (CỐT LÕI) ☐ C3. ≥ 2 quy tắc nghiệp vụ thật, không chỉ lưu và đọc dữ liệu. ☐ C4. ≥ 3 kiểm thử thuộc ba loại khác nhau, PASS bằng một lệnh. (CỐT LÕI) ☐ C5. Chạy lại lần hai không lỗi, không nhân đôi dữ liệu. (CỐT LÕI) ☐ C6. Đã commit tệp khoá phiên bản; .env.example đủ biến; .env thật không có trong repo. ☐ C7. Dữ liệu vào sai bị từ chối có thông báo rõ ràng, không sập, không lỗi thô.	[SE] ☐ S1 ≥ 2 thực thể đủ CRUD · ☐ S2 service không có req/res/mã HTTP ☐ S3 migration + seed từ CSDL trống · ☐ S4 ≥ 3 màn hình, đủ ba trạng thái [DA] ☐ D1 pipeline một lệnh, ba pha ba nơi · ☐ D2 ≥ 5 hàm kiểm chất lượng ra số ☐ D3 bảng reject có dòng, lý do viết cho người · ☐ D4 ≥ 2 biểu đồ có mốc so sánh [AI] ☐ A1 chia TRƯỚC khi fit · ☐ A2 baseline có số trong tệp chỉ số ☐ A3 chọn mô hình trên validation · ☐ A4 chỉ số theo nhãn + confusion matrix

Quy tắc quy đổi: đạt toàn bộ mục chung và mục track → tiêu chí 2 mức Tốt (8–10). Thiếu 1 mục → Khá. Thiếu 2–3 mục → Trung bình. Thiếu ≥ 4 mục HOẶC thiếu một mục CỐT LÕI (C1, C2, C4, C5) → Yếu. Mục nào không chứng minh được thì tính là KHÔNG ĐẠT.

Phép thử "thư mục trống": người khác có dựng lại được không?

PHÉP THỬ TỰ KIỂM

```
# 1. Mở một thư mục HOÀN TOÀN MỚI, không phải thư mục đang làm việc
cd /tmp && rm -rf thu-nghiem && mkdir thu-nghiem && cd thu-nghiem
# 2. Clone repo của theo tag
git clone --branch v0.2 <đường-dẫn-repo> .
# 3. Làm ĐÚNG theo README, từng bước, KHÔNG làm gì thêm
#    Phải gõ thêm một lệnh không có trong README
#    -> README chưa đủ, bổ sung lệnh đó ngay
# 4. Chạy bộ kiểm thử
npm test            # hoặc: pytest -q   -> phải PASS hết
# 5. Chạy lại lần hai (bước bị bỏ qua nhiều nhất)
<lệnh migration / pipeline / train>   # KHÔNG lỗi, KHÔNG nhân đôi
```

Ghi nhớ

- Đây chính là bước 1–3 và 6 trong quy trình chấm BT2: clone theo tag v0.2, làm đúng README.
- Phép thử luôn phát hiện ít nhất một thứ "chạy trên máy em": thiếu biến .env, thiếu seed, thiếu lệnh migration.
- Sinh viên làm phép thử này sớm luôn được điểm cao hơn ở mục C1 và C5.

KẾT THÚC BUỔI HỌC

Chốt buổi 7 và chuẩn bị cho buổi 8

Tự học, tự nghiên cứu

- Đọc tài liệu tự học buổi 7 theo track và ví dụ mã mẫu đã hoàn chỉnh.
- Dựng KHUNG module cốt lõi theo sơ đồ kiến trúc đã nộp ở BT1: tạo thư mục và file rỗng cho từng lớp, chưa cần viết logic.
- Kiểm tra lại môi trường phát triển vẫn chạy được (smoke test từ buổi 2).
- Tải bộ dữ liệu mẫu liên quan tới luồng từ VLU e-learning; đọc README của bộ dữ liệu.

Buổi 8 (thực hành, 3 tiết) — Workshop xây dựng module cốt lõi (Sprint 1)

- **NHẬN PHẢN HỒI BÀI TẬP 1**; điều chỉnh thiết kế nếu cần và ghi chú thay đổi.
- [SE] Tạo cơ sở dữ liệu; hiện thực CRUD cho ≥ 2 entity; viết ≥ 3 unit test.
- [DA] Script ETL: đọc nguồn → làm sạch → nạp staging; báo cáo số dòng trước và sau.
- [AI] Thu thập và làm sạch dataset; notebook EDA có thống kê mô tả và biểu đồ.
- Commit theo quy ước; cập nhật README sau mỗi mốc.